Deepdive Übungen

Wednesday, October 08, 2025 12:29 PM

Übung 1 - Statebackend

Erstellen einer backend.tf Datei
Konfigurieren des azurerm backends auf den vorhandenen Storage Account "autworkshop[1-20]" in der gleichnamigen Ressource gruppe
Anlegen der Umgebung mit Remote Backend

Übung 2 - Datentypen

2a - Listen

Öffne die terraform console
Prüfe die Variable user_list
Gebe die Variable aus: var.user_list
Verändere die Liste und spiele damit rum

2b - Sets

Öffne die terraform console
Prüfe die Variable user_set
Gebe die Variable aus: var.user_set
Verändere das Set und spiele damit rum

2c - Maps

Öffne die terraform console
Prüfe die Variable aca_jobs
Gebe die Variable aus: var.aca_jobs
Verändere das Set und spiele damit rum

2d - Validierungen

Erstelle Validations für:
storage name: Länge größer 3 aber kleiner 24
secret name: Nur lowercase und "-" erlaubt

Übung 3 - Schleifen

3a - for_each

Statt einem Container, sollen mehrere Container mit einem for_each aufgebaut werden
Die Namen der Container sollen mit einer Namensliste definierbar sein
Hinweis -> hier wird toset benötigt oder man übergibt direkt ein set
Zusatz (validation): liste muss manual-job oder event-job enthalten

3b - dynamic

Aktuell gibt es pro env_var ein env block. Ändere es zu einem dynamic block ab
Hinweis: hier kann man eine map eines Objektes nutzen

Übung 4 - Lifecycle

4a - ignore_changes

CanChangeMe var mit beliebigen value zu environment_variables hinzufügen
Manuel in Portal ändern
Apply durchführen
Variable im Portal ansehen -> Variable wird von tf wieder geändert
Lifecycle mit ignore_changes hinzufügen
Manuel oder in tfvars ändern
Beide Änderungen werden ignoriert

4b - precondition

Precondition hinzufügen, CanChangeMe darf nicht leer sein
Einmal einen Value übergeben, einmal leer lassen schauen was beim apply passiert

Übung Tests 1 - Unit Test

1a - Simpler Unit Test

Erstelle im tests/ Unterverzeichnis einen default test in der Datei “default.tftest.hcl”
Erstelle einen Unit test der die Variable “storage_name” übergibt.
Nutze hierfür einen “run” block, mit command = plan
Führe "terraform test” aus

Relevante Dokumentation:
https://developer.hashicorp.com/terraform/language/tests#run-blocks

1b - Test mit expect failures

Erstelle einen zweiten Test in der selben Datei
Dieser soll einen storage_name mit mehr als 24 Zeichen übergeben bekommen
Fange den Fehler mit expect_failures
Führe "terraform test" aus

Relevante Dokumentation:
https://developer.hashicorp.com/terraform/language/tests#expecting-failures

1c - Globale Variablen und asserts

Setze die Variable “job_name” Global auf den Wert “workshopXX”
Prüfe in einem Test mithilfe von “assert”, dass der Wert von azurerm_container_app_job.manual_job.name folgendem Wert entspricht “workshopXX-manual-job”
https://developer.hashicorp.com/terraform/language/tests#variables

Übung Tests 2 - Integration Test

Erstelle ein 2. test file “apply.tftest.hcl”
Erstelle einen test mit command = apply
Setze als Variable "rg_name" auf den wert "autworkshopXX"
Überprüfe, dass die Werte azurerm_storage_account.tfdeepdiveacc.resource_group_name und azurerm_resource_group.rg.name übereinstimmen
Prüfe ob azurerm_resource_group.rg.name mit der variable rg_name üvereinstimmt
Führe den test aus

Übung 8 - Statemanagement

8a - Moved

In state nach "azurerm_storage_account" suchen
Mit moved block storage account name ändern
An allen stellen an denen Ressource benötigt wird name ändern
Apply ausführen
Ausgabe prüfen

8b - Removed

Moved Block löschen
Wir wollen den Storage Account Container nicht mehr über Terraform verwalten
Storage acc container block auskommentieren
Removed block einfügen mit einer Lifecycle Konfiguration um destroy auf false zu setzen
Apply durchführen

8c - Import

Removed Block entfernen
Storage acc container block wieder aktivieren
Storage acc container über Import Block wieder in Terraform verfügbar machen
Apply durchführen